PEP 675: Arbitrary Literal String Type

Status: Final · Created: 30-Nov-2021 · Public domain / CC0

Abstract

There is currently no way to specify, using type annotations, that a function parameter can be of any literal string type. We have to specify a precise literal string type, such as Literal["foo"]. This PEP introduces a supertype of literal string types: LiteralString. This allows a function to accept arbitrary literal string types, such as Literal["foo"] or Literal["bar"].

Motivation

Powerful APIs that execute SQL or shell commands often recommend that they be invoked with literal strings, rather than arbitrary user controlled strings. There is no way to express this recommendation in the type system, however, meaning security vulnerabilities sometimes occur when developers fail to follow it. For example, a naive way to look up a user record from a database is to accept a user id and insert it into a predefined SQL query:

::

```
def query_user(conn: Connection, user_id: str) -> User:
    query = f"SELECT * FROM data WHERE user_id = {user_id}"
    conn.execute(query)
    ...  # Transform data to a User object and return it
query_user(conn, "user123")  # OK.
```

However, the user-controlled data user_id is being mixed with the SQL command string, which means a malicious user could run arbitrary SQL commands:

To prevent such SQL injection attacks, SQL APIs offer parameterized queries, which separate the executed query from user-controlled data and make it impossible to run arbitrary queries. For example, with sqlite3, our original function would be written safely as a query with parameters:

::

```
def query_user(conn: Connection, user_id: str) -> User:
    query = "SELECT * FROM data WHERE user_id = ?"
    conn.execute(query, (user_id,))
    ...
```

The problem is that there is no way to enforce this discipline. sqlite3's own documentation can only admonish the reader to not dynamically build the sql argument from external input; the API's authors cannot express that through the type system. Users can (and often do) still use a convenient f-string as before and leave their code vulnerable to SQL injection.

Existing tools, such as the popular security linter Bandit, attempt to detect unsafe external data used in SQL APIs, by inspecting the AST or by other semantic pattern-matching. These tools, however, preclude common idioms like storing a large multi-line query in a variable before executing it, adding literal string modifiers to the query based on some conditions, or transforming the query string using a function. (We survey existing tools in the Rejected Alternatives_ section.) For example, many tools will detect a false positive issue in this benign snippet:

::

```
def query_data(conn: Connection, user_id: str, limit: bool) -> None:
    query = """
        SELECT
            user.name,
            user.age
        FROM data
        WHERE user_id = ?
```

```
    """
    if limit:
        query += " LIMIT 1"
    conn.execute(query, (user_id,))
```

We want to forbid harmful execution of user-controlled data while still allowing benign idioms like the above and not requiring extra user work.

To meet this goal, we introduce the LiteralString type, which only accepts string values that are known to be made of literals. This is a generalization of the Literal["foo"] type from 586. A string of type LiteralString cannot contain user-controlled data. Thus, any API that only accepts LiteralString will be immune to injection vulnerabilities (with pragmatic limitations).

Since we want the sqlite3 execute method to disallow strings built with user input, we would make its typeshed stub accept a sql query that is of type LiteralString:

::

```
from typing import LiteralString
def execute(self, sql: LiteralString, parameters: Iterable[str] = ...) -> Cursor: ...
```

This successfully forbids our unsafe SQL example. The variable query below is inferred to have type str, since it is created from a format string using user_id, and cannot be passed to execute:

::

```
def query_user(conn: Connection, user_id: str) -> User:
    query = f"SELECT * FROM data WHERE user_id = {user_id}"
    conn.execute(query)  # Error: Expected LiteralString, got str.
    ...
```

The method remains flexible enough to allow our more complicated example:

::

```
def query_data(conn: Connection, user_id: str, limit: bool) -> None:
    # This is a literal string.
    query = """
        SELECT
            user.name,
            user.age
        FROM data
        WHERE user_id = ?
    """
    if limit:
        # Still has type LiteralString because we added a literal string.
        query += " LIMIT 1"
    conn.execute(query, (user_id,))  # OK
```

Notice that the user did not have to change their SQL code at all. The type checker was able to infer the literal string type and complain only in case of violations.

LiteralString is also useful in other cases where we want strict command-data separation, such as when building shell commands or when rendering a string into an HTML response without escaping (see Appendix A: Other Uses_). Overall, this combination of strictness and flexibility makes it easy to enforce safer API usage in sensitive code without burdening users.

Usage statistics

In a sample of open-source projects using sqlite3, we found that conn.execute was called ~67% of the time with a safe string literal and ~33% of the time with a potentially unsafe, local string variable. Using this PEP's literal string type along with a type checker would prevent the unsafe portion of that 33% of cases (ie. the ones where user controlled data is incorporated into the query), while seamlessly allowing the safe ones to remain.

Rationale

Firstly, why use *types* to prevent security vulnerabilities?

Warning users in documentation is insufficient - most users either never see these warnings or ignore them. Using an existing dynamic or static analysis approach is too restrictive - these prevent natural idioms, as we saw in the Motivation_ section (and will discuss more extensively in the Rejected Alternatives_ section). The typing-based approach in this PEP strikes a user-friendly balance between strictness and flexibility.

Runtime approaches do not work because, at runtime, the query string is a plain str. While we could prevent some exploits using heuristics, such as regex-filtering for obviously malicious payloads, there will always be a way to work around them (perfectly distinguishing good and bad queries reduces to the halting problem).

Static approaches, such as checking the AST to see if the query string is a literal string expression, cannot tell when a string is assigned to an intermediate variable or when it is transformed by a benign function. This makes them overly restrictive.

The type checker, surprisingly, does better than both because it has access to information not available in the runtime or static analysis approaches. Specifically, the type checker can tell us whether an expression has a literal string type, say Literal["foo"]. The type checker already propagates types across variable assignments or function calls.

In the current type system itself, if the SQL or shell command execution function only accepted three possible input strings, our job would be done. We would just say:

::

```
def execute(query: Literal["foo", "bar", "baz"]) -> None: ...
```

But, of course, execute can accept *any* possible query. How do we ensure that the query does not contain an arbitrary, user-controlled string?

We want to specify that the value must be of some type Literal[<...>] where <...> is some string. This is what LiteralString represents. LiteralString is the "supertype" of all literal string types. In effect, this PEP just introduces a type in the type hierarchy between Literal["foo"] and str. Any particular literal string, such as Literal["foo"] or Literal["bar"], is compatible with LiteralString, but not the other way around. The "supertype" of LiteralString itself is str. So, LiteralString is compatible with str, but not the other way around.

Note that a Union of literal types is naturally compatible with LiteralString because each element of the Union is individually compatible with LiteralString. So, Literal["foo", "bar"] is compatible with LiteralString.

However, recall that we don't just want to represent exact literal queries. We also want to support composition of two literal strings, such as query + " LIMIT 1". This too is possible with the above concept. If x and y are two values of type LiteralString, then x + y will also be of type compatible with LiteralString. We can reason about this by looking at specific instances such as Literal["foo"] and Literal["bar"]; the value of the added string x + y can only be "foobar", which has type Literal["foobar"] and is thus compatible with LiteralString. The same reasoning applies when x and y are unions of literal types; the result of pairwise adding any two literal types from x and y respectively is a literal type, which means that the overall result is a Union of literal types and is thus compatible with LiteralString.

In this way, we are able to leverage Python's concept of a Literal string type to specify that our API can only accept strings that are known to be constructed from literals. More specific details follow in the remaining sections.

Specification

Runtime Behavior

We propose adding LiteralString to typing.py, with an implementation similar to typing.NoReturn.

Note that LiteralString is a special form used solely for type checking. There is no expression for which type(<expr>) will produce LiteralString at runtime. So, we do not specify in the implementation that it is a subclass of str.

Valid Locations for LiteralString

LiteralString can be used where any other type can be used:

::

```
variable_annotation: LiteralString
def my_function(literal_string: LiteralString) -> LiteralString: ...
class Foo:
    my_attribute: LiteralString
type_argument: List[LiteralString]
T = TypeVar("T", bound=LiteralString)
```

It cannot be nested within unions of Literal types:

::

```
bad_union: Literal["hello", LiteralString]  # Not OK
bad_nesting: Literal[LiteralString]  # Not OK
```

Type Inference

Inferring LiteralString

Any literal string type is compatible with LiteralString. For example, x: LiteralString = "foo" is valid because "foo" is inferred to be of type Literal["foo"].

As per the Rationale_, we also infer LiteralString in the following cases:

+ Addition: x + y is of type LiteralString if both x and y are compatible with LiteralString.

+ Joining: sep.join(xs) is of type LiteralString if sep's type is compatible with LiteralString and xs's type is compatible with Iterable[LiteralString].

+ In-place addition: If s has type LiteralString and x has type compatible with LiteralString, then s += x preserves s's type as LiteralString.

+ String formatting: An f-string has type LiteralString if and only if its constituent expressions are literal strings. s.format(...) has type LiteralString if and only if s and the arguments have types compatible with LiteralString.

+ Literal-preserving methods: In Appendix C, we have provided an exhaustive list of str methods that preserve the LiteralString type.

In all other cases, if one or more of the composed values has a non-literal type str, the composition of types will have type str. For example, if s has type str, then "hello" + s has type str. This matches the pre-existing behavior of type checkers.

LiteralString is compatible with the type str. It inherits all methods from str. So, if we have a variable s of type LiteralString, it is safe to write s.startswith("hello").

Some type checkers refine the type of a string when doing an equality check:

::

```
def foo(s: str) -> None:
```

```
    if s == "bar":
        reveal_type(s)  # => Literal["bar"]
```

Such a refined type in the if-block is also compatible with LiteralString because its type is Literal["bar"].

Examples

See the examples below to help clarify the above rules:

::

```
literal_string: LiteralString
s: str = literal_string  # OK
literal_string: LiteralString = s  # Error: Expected LiteralString, got str.
literal_string: LiteralString = "hello"  # OK
```

Addition of literal strings:

::

```
def expect_literal_string(s: LiteralString) -> None: ...
expect_literal_string("foo" + "bar")  # OK
expect_literal_string(literal_string + "bar")  # OK
literal_string2: LiteralString
expect_literal_string(literal_string + literal_string2)  # OK
plain_string: str
expect_literal_string(literal_string + plain_string)  # Not OK.
```

Join using literal strings:

::

```
expect_literal_string(",".join(["foo", "bar"]))  # OK
expect_literal_string(literal_string.join(["foo", "bar"]))  # OK
expect_literal_string(literal_string.join([literal_string, literal_string2]))  # OK
xs: List[LiteralString]
expect_literal_string(literal_string.join(xs)) # OK
expect_literal_string(plain_string.join([literal_string, literal_string2]))
# Not OK because the separator has type 'str'.
```

In-place addition using literal strings:

::

```
literal_string += "foo"  # OK
literal_string += literal_string2  # OK
literal_string += plain_string # Not OK
```

Format strings using literal strings:

::

```
literal_name: LiteralString
expect_literal_string(f"hello {literal_name}")
# OK because it is composed from literal strings.
expect_literal_string("hello {}".format(literal_name))  # OK
expect_literal_string(f"hello")  # OK
username: str
expect_literal_string(f"hello {username}")
# NOT OK. The format-string is constructed from 'username',
# which has type 'str'.
expect_literal_string("hello {}".format(username))  # Not OK
```

Other literal types, such as literal integers, are not compatible with LiteralString:

::

```
some_int: int
expect_literal_string(some_int)  # Error: Expected LiteralString, got int.
literal_one: Literal[1] = 1
expect_literal_string(literal_one)  # Error: Expected LiteralString, got Literal[1].
```

We can call functions on literal strings:

::

```
def add_limit(query: LiteralString) -> LiteralString:
    return query + " LIMIT = 1"
def my_query(query: LiteralString, user_id: str) -> None:
    sql_connection().execute(add_limit(query), (user_id,))  # OK
```

Conditional statements and expressions work as expected:

::

```
def return_literal_string() -> LiteralString:
    return "foo" if condition1() else "bar"  # OK
def return_literal_str2(literal_string: LiteralString) -> LiteralString:
    return "foo" if condition1() else literal_string  # OK
def return_literal_str3() -> LiteralString:
    if condition1():
        result: Literal["foo"] = "foo"
    else:
        result: LiteralString = "bar"
    return result  # OK
```

Interaction with TypeVars and Generics

TypeVars can be bound to LiteralString:

::

```
from typing import Literal, LiteralString, TypeVar
TLiteral = TypeVar("TLiteral", bound=LiteralString)
def literal_identity(s: TLiteral) -> TLiteral:
    return s
hello: Literal["hello"] = "hello"
y = literal_identity(hello)
reveal_type(y)  # => Literal["hello"]
s: LiteralString
y2 = literal_identity(s)
reveal_type(y2)  # => LiteralString
s_error: str
literal_identity(s_error)
# Error: Expected TLiteral (bound to LiteralString), got str.
```

LiteralString can be used as a type argument for generic classes:

::

```
class Container(Generic[T]):
    def __init__(self, value: T) -> None:
        self.value = value
literal_string: LiteralString = "hello"
x: Container[LiteralString] = Container(literal_string)  # OK
s: str
x_error: Container[LiteralString] = Container(s)  # Not OK
```

Standard containers like List work as expected:

::

```
xs: List[LiteralString] = ["foo", "bar", "baz"]
```

Interactions with Overloads

Literal strings and overloads do not need to interact in a special way: the existing rules work fine. LiteralString can be used as a fallback overload where a specific Literal["foo"] type does not match:

::

```
@overload
def foo(x: Literal["foo"]) -> int: ...
@overload
def foo(x: LiteralString) -> bool: ...
@overload
def foo(x: str) -> str: ...
x1: int = foo("foo")  # First overload.
x2: bool = foo("bar")  # Second overload.
s: str
x3: str = foo(s)  # Third overload.
```

Backwards Compatibility

We propose adding typing_extensions.LiteralString for use in earlier Python versions.

As PEP 586 mentions, type checkers "should feel free to experiment with more sophisticated inference techniques". So, if the type checker infers a literal string type for an unannotated variable that is initialized with a literal string, the following example should be OK:

::

```
x = "hello"
expect_literal_string(x)
# OK, because x is inferred to have type 'Literal["hello"]'.
```

This enables precise type checking of idiomatic SQL query code without annotating the code at all (as seen in the Motivation_ section example).

However, like 586, this PEP does not mandate the above inference strategy. In case the type checker doesn't infer x to have type Literal["hello"], users can aid the type checker by explicitly annotating it as x: LiteralString:

::

```
x: LiteralString = "hello"
expect_literal_string(x)
```

Rejected Alternatives

Why not use tool X?

Tools to catch issues such as SQL injection seem to come in three flavors: AST based, function level analysis, and taint flow analysis.

AST-based tools: Bandit has a plugin to warn when SQL queries are not literal strings. The problem is that many perfectly safe SQL queries are dynamically built out of string literals, as shown in the Motivation_ section. At the AST level, the resultant SQL query is not going to appear as a string literal anymore and is thus indistinguishable from a potentially malicious string. To use these tools would require significantly restricting developers' ability to build SQL queries. LiteralString can

provide similar safety guarantees with fewer restrictions.

Semgrep and pyanalyze: Semgrep supports a more sophisticated function level analysis, including constant propagation within a function. This allows us to prevent injection attacks while permitting some forms of safe dynamic SQL queries within a function. pyanalyze has a similar extension. But neither handles function calls that construct and return safe SQL queries. For example, in the code sample below, build_insert_query is a helper function to create a query that inserts multiple values into the corresponding columns. Semgrep and pyanalyze forbid this natural usage whereas LiteralString handles it with no burden on the programmer:

::

```
def build_insert_query(
    table: LiteralString
    insert_columns: Iterable[LiteralString],
) -> LiteralString:
    sql = "INSERT INTO " + table
    column_clause = ", ".join(insert_columns)
    value_clause = ", ".join(["?"] * len(insert_columns))
    sql += f" ({column_clause}) VALUES ({value_clause})"
    return sql
def insert_data(
    conn: Connection,
    kvs_to_insert: Dict[LiteralString, str]
) -> None:
    query = build_insert_query("data", kvs_to_insert.keys())
    conn.execute(query, kvs_to_insert.values())
# Example usage
data_to_insert = {
    "column_1": value_1,  # Note: values are not literals
    "column_2": value_2,
    "column_3": value_3,
}
insert_data(conn, data_to_insert)
```

Taint flow analysis: Tools such as Pysa or CodeQL are capable of tracking data flowing from a user controlled input into a SQL query. These tools are powerful but involve considerable overhead in setting up the tool in CI, defining "taint" sinks and sources, and teaching developers how to use them. They also usually take longer to run than a type checker (minutes instead of seconds), which means feedback is not immediate. Finally, they move the burden of preventing vulnerabilities on to library users instead of allowing the libraries themselves to specify precisely how their APIs must be called (as is possible with LiteralString).

One final reason to prefer using a new type over a dedicated tool is that type checkers are more widely used than dedicated security tooling; for example, MyPy was downloaded over 7 million times in Jan 2022 vs less than 2 million times for Bandit. Having security protections built right into type checkers will mean that more developers benefit from them.

Why not use a NewType for str?

Any API for which LiteralString would be suitable could instead be updated to accept a different type created within the Python type system, such as NewType("SafeSQL", str):

::

```
SafeSQL = NewType("SafeSQL", str)
def execute(self, sql: SafeSQL, parameters: Iterable[str] = ...) -> Cursor: ...
execute(SafeSQL("SELECT * FROM data WHERE user_id = ?"), user_id)  # OK
user_query: str
execute(user_query)  # Error: Expected SafeSQL, got str.
```

Having to create a new type to call an API might give some developers pause and encourage more caution, but it doesn't guarantee that developers won't just turn a user controlled string into the new type, and pass it into the modified API anyway:

::

```
query = f"SELECT * FROM data WHERE user_id = f{user_id}"
execute(SafeSQL(query))  # No error!
```

We are back to square one with the problem of preventing arbitrary inputs to SafeSQL. This is not a theoretical concern either. Django uses the above approach with SafeString and mark_safe. Issues such as CVE-2020-13596 show how this technique can fail.

Also note that this requires invasive changes to the source code (wrapping the query with SafeSQL) whereas LiteralString requires no such changes. Users can remain oblivious to it as long as they pass in literal strings to sensitive APIs.

Why not try to emulate Trusted Types?

Trusted Types is a W3C specification for preventing DOM-based Cross Site Scripting (XSS). XSS occurs when dangerous browser APIs accept raw user-controlled strings. The specification modifies these APIs to accept only the "Trusted Types" returned by designated sanitizing functions. These sanitizing functions must take in a potentially malicious string and validate it or render it benign somehow, for example by verifying that it is a valid URL or HTML-encoding it.

It can be tempting to assume porting the concept of Trusted Types to Python could solve the problem. The fundamental difference, however, is that the output of a Trusted Types sanitizer is usually intended *to not be executable code*. Thus it's easy to HTML encode the input, strip out dangerous tags, or otherwise render it inert. With a SQL query or shell command, the end result *still needs to be executable code*. There is no way to write a sanitizer that can reliably figure out which parts of an input string are benign and which ones are potentially malicious.

Runtime Checkable LiteralString

The LiteralString concept could be extended beyond static type checking to be a runtime checkable property of str objects. This would provide some benefits, such as allowing frameworks to raise errors on dynamic strings. Such runtime errors would be a more robust defense mechanism than type errors, which can potentially be suppressed, ignored, or never even seen if the author does not use a type checker.

This extension to the LiteralString concept would dramatically increase the scope of the proposal by requiring changes to one of the most fundamental types in Python. While runtime taint checking on strings, similar to Perl's taint, has been considered and attempted in the past, and others may consider it in the future, such extensions are out of scope for this PEP.

Rejected Names

We considered a variety of names for the literal string type and solicited ideas on typing-sig. Some notable alternatives were:

+ Literal[str]: This is a natural extension of the Literal["foo"] type name, but typing-sig objected that users could mistake this for the literal type of the str class.

+ LiteralStr: This is shorter than LiteralString but looks weird to the PEP authors.

+ LiteralDerivedString: This (along with MadeFromLiteralString) best captures the technical meaning of the type. It represents not just the type of literal expressions, such as "foo", but also that of expressions composed from literals, such as "foo" + "bar". However, both names seem wordy.

+ StringLiteral: Users might confuse this with the existing concept of "string literals" where the string exists as a syntactic token in the source code, whereas our concept is more general.

+ SafeString: While this comes close to our intended meaning, it may mislead users into thinking that the string has been sanitized in some way, perhaps by escaping HTML tags or shell-related special characters.

+ ConstantStr: This does not capture the idea of composing literal strings.

+ StaticStr: This suggests that the string is statically computable, i.e., computable without running the program, which is not true. The literal string may vary based on runtime flags, as seen in the Motivation_ examples.

+ LiteralOnly[str]: This has the advantage of being extensible to other literal types, such as bytes or int. However, we did not find the extensibility worth the loss of readability.

Overall, there was no clear winner on typing-sig over a long period, so we decided to tip the scales in favor of LiteralString.

LiteralBytes

We could generalize literal byte types, such as Literal[b"foo"], to LiteralBytes. However, literal byte types are used much less frequently than literal string types and we did not find much user demand for LiteralBytes, so we decided not to include it in this PEP. Others may, however, consider it in future PEPs.

Reference Implementation

This is implemented in Pyre v0.9.8 and is actively being used.

The implementation simply extends the type checker with LiteralString as a supertype of literal string types.

To support composition via addition, join, etc., it was sufficient to overload the stubs for str in Pyre's copy of typeshed.

Appendix A: Other Uses

To simplify the discussion and require minimal security knowledge, we focused on SQL injections throughout the PEP. LiteralString, however, can also be used to prevent many other kinds of injection vulnerabilities.

Command Injection

APIs such as subprocess.run accept a string which can be run as a shell command:

::

```
subprocess.run(f"echo 'Hello {name}'", shell=True)
```

If user-controlled data is included in the command string, the code is vulnerable to "command injection"; i.e., an attacker can run malicious commands. For example, a value of ' && rm -rf / # would result in the following destructive command being run:

::

```
echo 'Hello ' && rm -rf / #'
```

This vulnerability could be prevented by updating run to only accept LiteralString when used in shell=True mode. Here is one simplified stub:

::

```
def run(command: LiteralString, *args: str, shell: bool=...): ...
```

Cross Site Scripting (XSS)

Most popular Python web frameworks, such as Django, use a templating engine to produce HTML from user data. These templating languages auto-escape user data before inserting it into the HTML template and thus prevent cross site scripting (XSS) vulnerabilities.

But a common way to bypass auto-escaping and render HTML as-is is to use functions like mark_safe in Django or do_mark_safe in Jinja2, which cause XSS vulnerabilities:

This vulnerability could be prevented by updating mark_safe to only accept LiteralString:

Server Side Template Injection (SSTI)

Templating frameworks, such as Jinja, allow Python expressions which will be evaluated and substituted into the rendered result:

::

```
template_str = "There are {{ len(values) }} values: {{ values }}"
template = jinja2.Template(template_str)
template.render(values=[1, 2])
# Result: "There are 2 values: [1, 2]"
```

If an attacker controls all or part of the template string, they can insert expressions which execute arbitrary code and compromise the application:

Template injection exploits like this could be prevented by updating the Template API to only accept LiteralString:

Logging Format String Injection

Logging frameworks often allow their input strings to contain formatting directives. At its worst, allowing users to control the logged string has led to CVE-2021-44228 (colloquially known as log4shell), which has been described as the "most critical vulnerability of the last decade". While no Python frameworks are currently known to be vulnerable to a similar attack, the built-in logging framework does provide formatting options which are vulnerable to Denial of Service attacks from externally controlled logging strings. The following example illustrates a simple denial of service scenario:

This kind of attack could be prevented by requiring that the format string passed to the logger be a LiteralString and that all externally controlled data be passed separately as arguments (as proposed in Issue 46200):

Appendix B: Limitations

There are a number of ways LiteralString could still fail to prevent users from passing strings built from non-literal data to an API:

1. If the developer does not use a type checker or does not add type annotations, then violations will go uncaught.

2. cast(LiteralString, non_literal_string) could be used to lie to the type checker and allow a dynamic string value to masquerade as a LiteralString. The same goes for a variable that has type Any.

3. Comments such as # type: ignore could be used to ignore warnings about non-literal strings.

4. Trivial functions could be constructed to convert a str to a LiteralString:

We could mitigate the above using linting, code review, etc., but ultimately a clever, malicious developer attempting to circumvent the protections offered by LiteralString will always succeed. The important thing to remember is that LiteralString is not intended to protect against *malicious* developers; it is meant to protect against benign developers accidentally using sensitive APIs in a dangerous way (without getting in their way otherwise).

Without LiteralString, the best enforcement tool API authors have is documentation, which is easily ignored and often not seen. With LiteralString, API misuse requires conscious thought and artifacts in the code that reviewers and future developers can notice.

Appendix C: str methods that preserve LiteralString

The str class has several methods that would benefit from LiteralString. For example, users might expect "hello".capitalize() to have the type LiteralString similar to the other examples we have seen in the Inferring LiteralString section. Inferring the type LiteralString is correct because the string is not an arbitrary user-supplied string - we know that it has the type Literal["HELLO"], which is compatible with LiteralString. In other words, the capitalize method preserves the LiteralString type. There are several other str methods that preserve LiteralString.

We propose updating the stub for str in typeshed so that the methods are overloaded with the LiteralString-preserving versions. This means type checkers do not have to hardcode LiteralString behavior for each method. It also lets us easily support new methods in the future by updating the typeshed stub.

For example, to preserve literal types for the capitalize method, we would change the stub as below:

::

```
# before
def capitalize(self) -> str: ...
# after
@overload
def capitalize(self: LiteralString) -> LiteralString: ...
@overload
def capitalize(self) -> str: ...
```

The downside of changing the str stub is that the stub becomes more complicated and can make error messages harder to understand. Type checkers may need to special-case str to make error messages understandable for users.

Below is an exhaustive list of str methods which, when called with arguments of type LiteralString, must be treated as returning a LiteralString. If this PEP is accepted, we will update these method signatures in typeshed:

::

```
@overload
def capitalize(self: LiteralString) -> LiteralString: ...
@overload
def capitalize(self) -> str: ...
@overload
def casefold(self: LiteralString) -> LiteralString: ...
@overload
def casefold(self) -> str: ...
@overload
def center(self: LiteralString, __width: SupportsIndex, __fillchar: LiteralString = ...) -> Lite
@overload
def center(self, __width: SupportsIndex, __fillchar: str = ...) -> str: ...
if sys.version_info >= (3, 8):
    @overload
    def expandtabs(self: LiteralString, tabsize: SupportsIndex = ...) -> LiteralString: ...
    @overload
    def expandtabs(self, tabsize: SupportsIndex = ...) -> str: ...
else:
    @overload
    def expandtabs(self: LiteralString, tabsize: int = ...) -> LiteralString: ...
    @overload
    def expandtabs(self, tabsize: int = ...) -> str: ...
@overload
def format(self: LiteralString, *args: LiteralString, **kwargs: LiteralString) -> LiteralString:
@overload
def format(self, *args: str, **kwargs: str) -> str: ...
@overload
```

```
def join(self: LiteralString, __iterable: Iterable[LiteralString]) -> LiteralString: ...
@overload
def join(self, __iterable: Iterable[str]) -> str: ...
@overload
def ljust(self: LiteralString, __width: SupportsIndex, __fillchar: LiteralString = ...) -> Litera
@overload
def ljust(self, __width: SupportsIndex, __fillchar: str = ...) -> str: ...
@overload
def lower(self: LiteralString) -> LiteralString: ...
@overload
def lower(self) -> LiteralString: ...
@overload
def lstrip(self: LiteralString, __chars: LiteralString | None = ...) -> LiteralString: ...
@overload
def lstrip(self, __chars: str | None = ...) -> str: ...
@overload
def partition(self: LiteralString, __sep: LiteralString) -> tuple[LiteralString, LiteralString, L
@overload
def partition(self, __sep: str) -> tuple[str, str, str]: ...
@overload
def replace(self: LiteralString, __old: LiteralString, __new: LiteralString, __count: SupportsInd
@overload
def replace(self, __old: str, __new: str, __count: SupportsIndex = ...) -> str: ...
if sys.version_info >= (3, 9):
    @overload
    def removeprefix(self: LiteralString, __prefix: LiteralString) -> LiteralString: ...
    @overload
    def removeprefix(self, __prefix: str) -> str: ...
    @overload
    def removesuffix(self: LiteralString, __suffix: LiteralString) -> LiteralString: ...
    @overload
    def removesuffix(self, __suffix: str) -> str: ...
@overload
def rjust(self: LiteralString, __width: SupportsIndex, __fillchar: LiteralString = ...) -> Litera
@overload
def rjust(self, __width: SupportsIndex, __fillchar: str = ...) -> str: ...
@overload
def rpartition(self: LiteralString, __sep: LiteralString) -> tuple[LiteralString, LiteralString,
@overload
def rpartition(self, __sep: str) -> tuple[str, str, str]: ...
@overload
def rsplit(self: LiteralString, sep: LiteralString | None = ..., maxsplit: SupportsIndex = ...) -
@overload
def rsplit(self, sep: str | None = ..., maxsplit: SupportsIndex = ...) -> list[str]: ...
@overload
def rstrip(self: LiteralString, __chars: LiteralString | None = ...) -> LiteralString: ...
@overload
def rstrip(self, __chars: str | None = ...) -> str: ...
@overload
def split(self: LiteralString, sep: LiteralString | None = ..., maxsplit: SupportsIndex = ...) ->
@overload
def split(self, sep: str | None = ..., maxsplit: SupportsIndex = ...) -> list[str]: ...
@overload
def splitlines(self: LiteralString, keepends: bool = ...) -> list[LiteralString]: ...
@overload
def splitlines(self, keepends: bool = ...) -> list[str]: ...
@overload
def strip(self: LiteralString, __chars: LiteralString | None = ...) -> LiteralString: ...
@overload
def strip(self, __chars: str | None = ...) -> str: ...
@overload
def swapcase(self: LiteralString) -> LiteralString: ...
@overload
def swapcase(self) -> str: ...
@overload
def title(self: LiteralString) -> LiteralString: ...
@overload
def title(self) -> str: ...
@overload
def upper(self: LiteralString) -> LiteralString: ...
@overload
def upper(self) -> str: ...
@overload
def zfill(self: LiteralString, __width: SupportsIndex) -> LiteralString: ...
@overload
def zfill(self, __width: SupportsIndex) -> str: ...
@overload
```

```
def __add__(self: LiteralString, __s: LiteralString) -> LiteralString: ...
@overload
def __add__(self, __s: str) -> str: ...
@overload
def __iter__(self: LiteralString) -> Iterator[str]: ...
@overload
def __iter__(self) -> Iterator[str]: ...
@overload
def __mod__(self: LiteralString, __x: Union[LiteralString, Tuple[LiteralString, ...]]) -> str: .
@overload
def __mod__(self, __x: Union[str, Tuple[str, ...]]) -> str: ...
@overload
def __mul__(self: LiteralString, __n: SupportsIndex) -> LiteralString: ...
@overload
def __mul__(self, __n: SupportsIndex) -> str: ...
@overload
def __repr__(self: LiteralString) -> LiteralString: ...
@overload
def __repr__(self) -> str: ...
@overload
def __rmul__(self: LiteralString, n: SupportsIndex) -> LiteralString: ...
@overload
def __rmul__(self, n: SupportsIndex) -> str: ...
@overload
def __str__(self: LiteralString) -> LiteralString: ...
@overload
def __str__(self) -> str: ...
```

Appendix D: Guidelines for using LiteralString in Stubs

Libraries that do not contain type annotations within their source may specify type stubs in Typeshed. Libraries written in other languages, such as those for machine learning, may also provide Python type stubs. This means the type checker cannot verify that the type annotations match the source code and must trust the type stub. Thus, authors of type stubs need to be careful when using LiteralString, since a function may falsely appear to be safe when it is not.

We recommend the following guidelines for using LiteralString in stubs:

+ If the stub is for a pure function, we recommend using LiteralString in the return type of the function or of its overloads only if all the corresponding parameters have literal types (i.e., LiteralString or Literal["a", "b"]).

::

```
# OK
@overload
def my_transform(x: LiteralString, y: Literal["a", "b"]) -> LiteralString: ...
@overload
def my_transform(x: str, y: str) -> str: ...
# Not OK
@overload
def my_transform(x: LiteralString, y: str) -> LiteralString: ...
@overload
def my_transform(x: str, y: str) -> str: ...
```

+ If the stub is for a staticmethod, we recommend the same guideline as above.

+ If the stub is for any other kind of method, we recommend against using LiteralString in the return type of the method or any of its overloads. This is because, even if all the explicit parameters have type LiteralString, the object itself may be created using user data and thus the return type may be user-controlled.

+ If the stub is for a class attribute or global variable, we also recommend against using LiteralString because the untyped code may write arbitrary values to the attribute.

However, we leave the final call to the library author. They may use LiteralString if they feel confident that the string returned by the method or function or the string stored in the attribute is guaranteed to have a literal type - i.e., the string is created by applying only literal-preserving str operations to a string literal.

Note that these guidelines do not apply to inline type annotations since the type checker can verify that, say, a method returning LiteralString does in fact return an expression of that type.

Resources

Literal String Types in Scala

Scala uses Singleton as the supertype for singleton types, which includes literal string types, such as "foo". Singleton is Scala's generalized analogue of this PEP's LiteralString.

Tamer Abdulradi showed how Scala's literal string types can be used for "Preventing SQL injection at compile time", Scala Days talk Literal types: What are they good for? (slides 52 to 68).

Thanks

Thanks to the following people for their feedback on the PEP:

Edward Qiu, Jia Chen, Shannon Zhu, Gregory P. Smith, ■■■■■■ ■■■■■■■■, CAM Gerlach, Arie Bovenberg, David Foster, and Shengye Wan

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.